

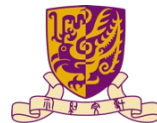
2026 ACM International Conference on the Foundations of Software Engineering (FSE 2026)

Bringing Managed Language Support to WebAssembly with External Library Linking

Shuyao Jiang¹, Ruiying Zeng², Yangfan Zhou², Michael R. Lyu¹

1. Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong, China

2. College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China



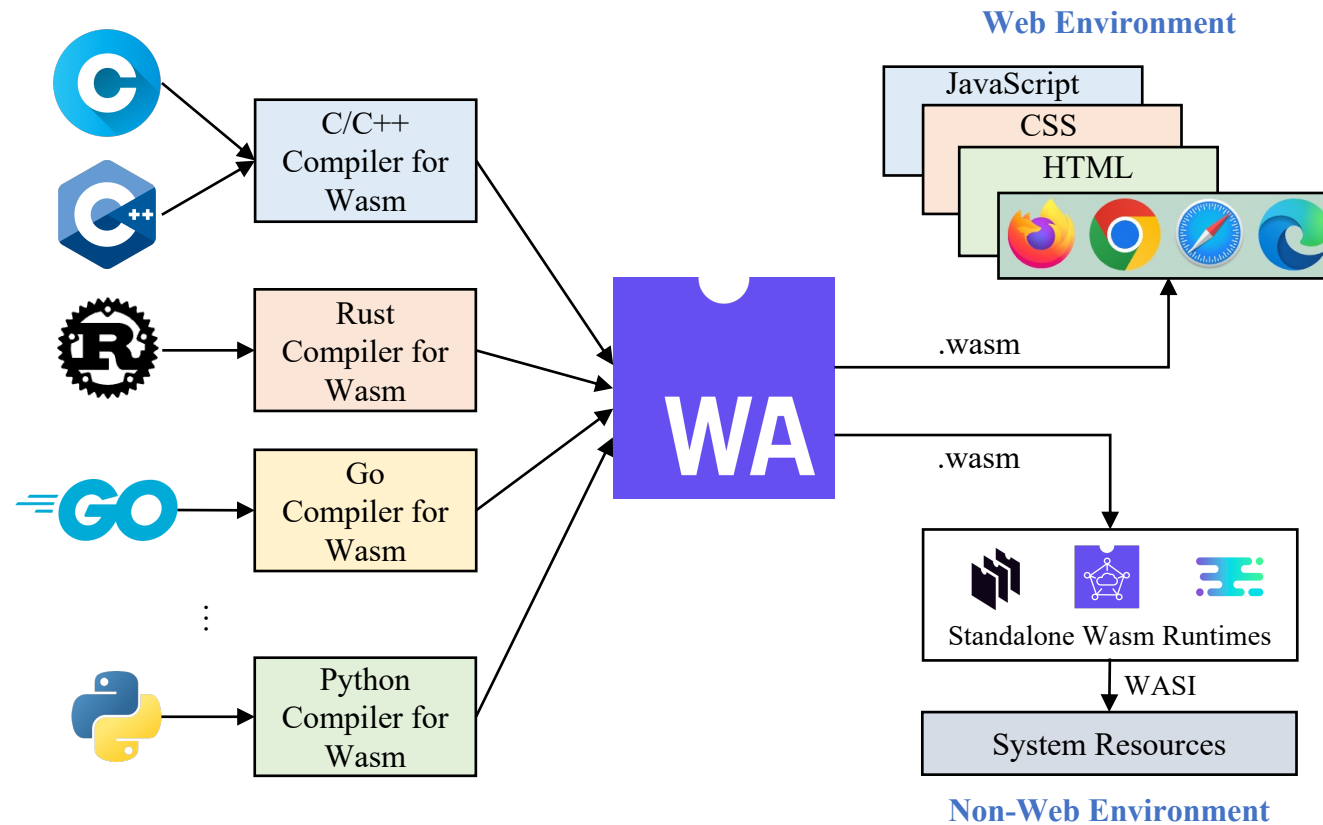
香港中文大學
The Chinese University of Hong Kong



復旦大學
FUDAN UNIVERSITY

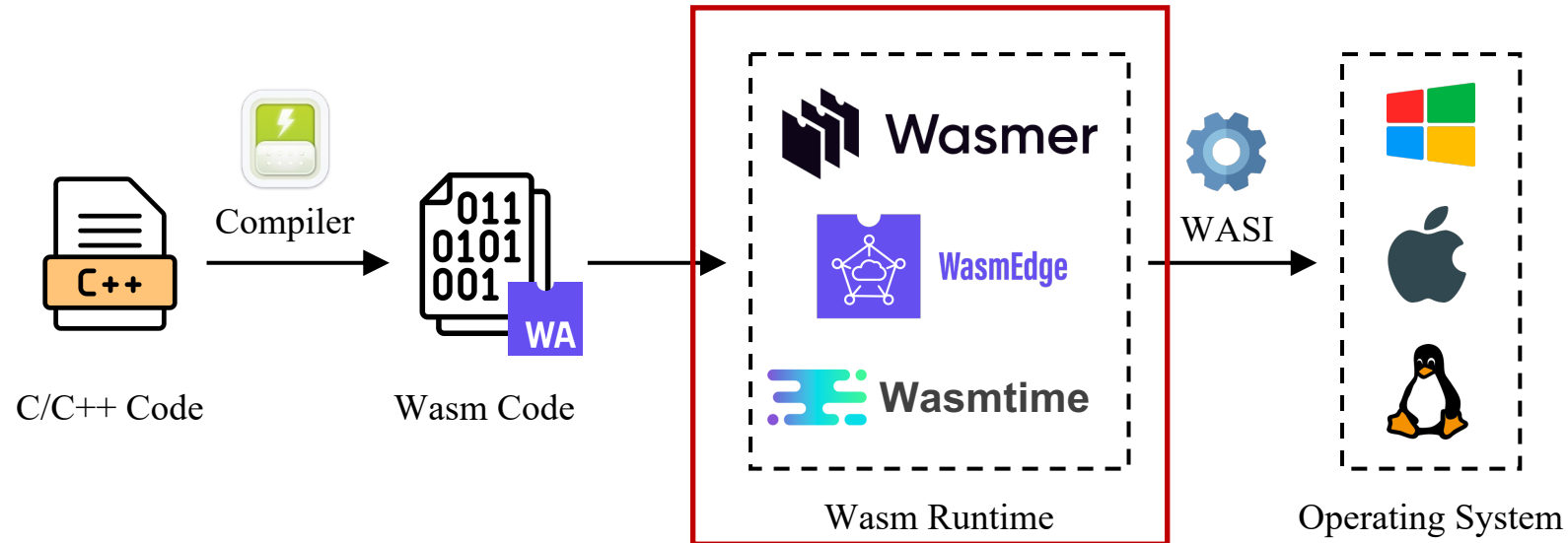
WebAssembly (Wasm)

- A binary instruction format
 - Designed as a portable **compilation target** for programming languages
 - For multi-language and sandboxed execution



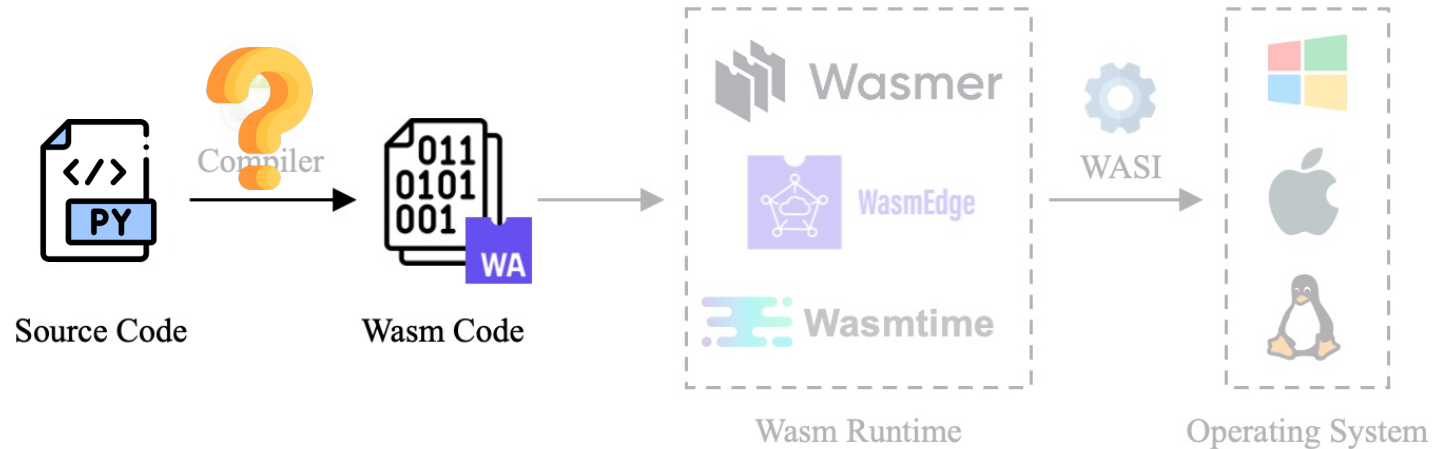
Wasm Application Workflow

- Key Component: **Wasm Runtime**
 - Translate Wasm binary instructions to native machine code
 - Interact with OS through *WebAssembly System Interface (WASI)*



Multi-language Support for Wasm

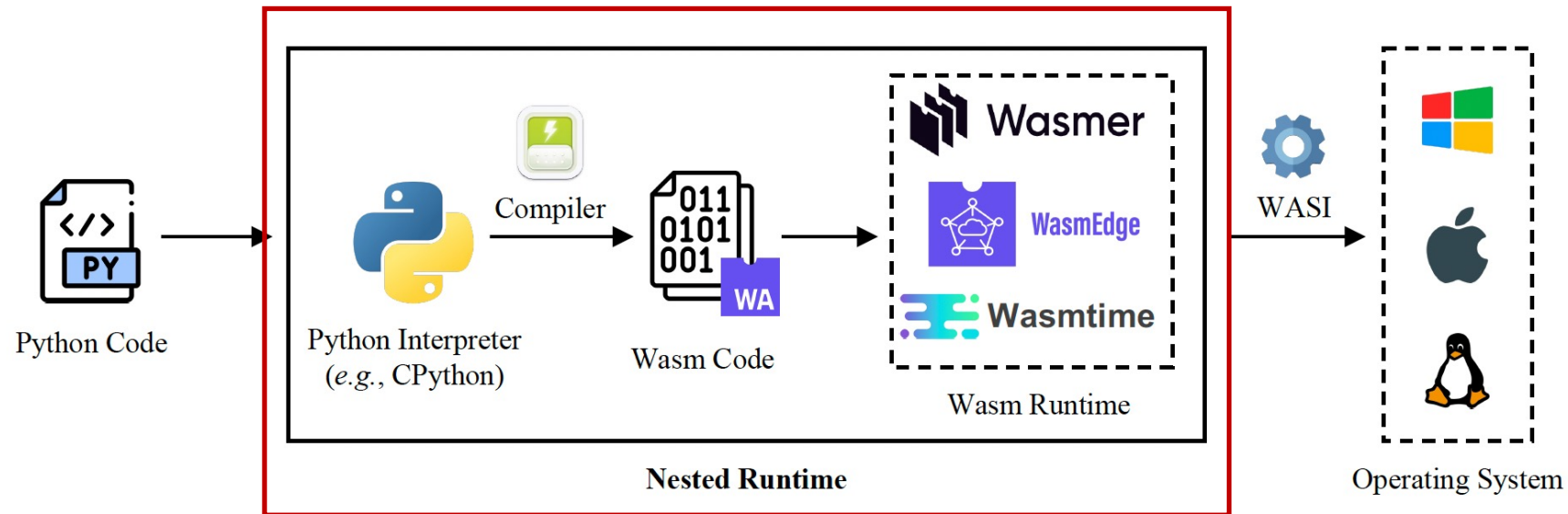
- **Well support:** Typical **compiled languages**
 - C, C++, Rust
- **Poor support:** Most **managed languages**
 - Python, Java, JavaScript



Key challenge: How to handle the **managed runtime** of the source language?

Existing Solution: Runtime Nesting

- Compile the managed runtime into Wasm format
- Run the source program on the **Wasm-formatted runtime**



Non-extensible

Poor Performance

Limited Support



Existing Solution: Runtime Nesting

- Compile the managed runtime into Wasm format
- Run the source program on the **Wasm-formatted runtime**

Can we design a **more efficient** language support mechanism for Wasm?



Non-extensible

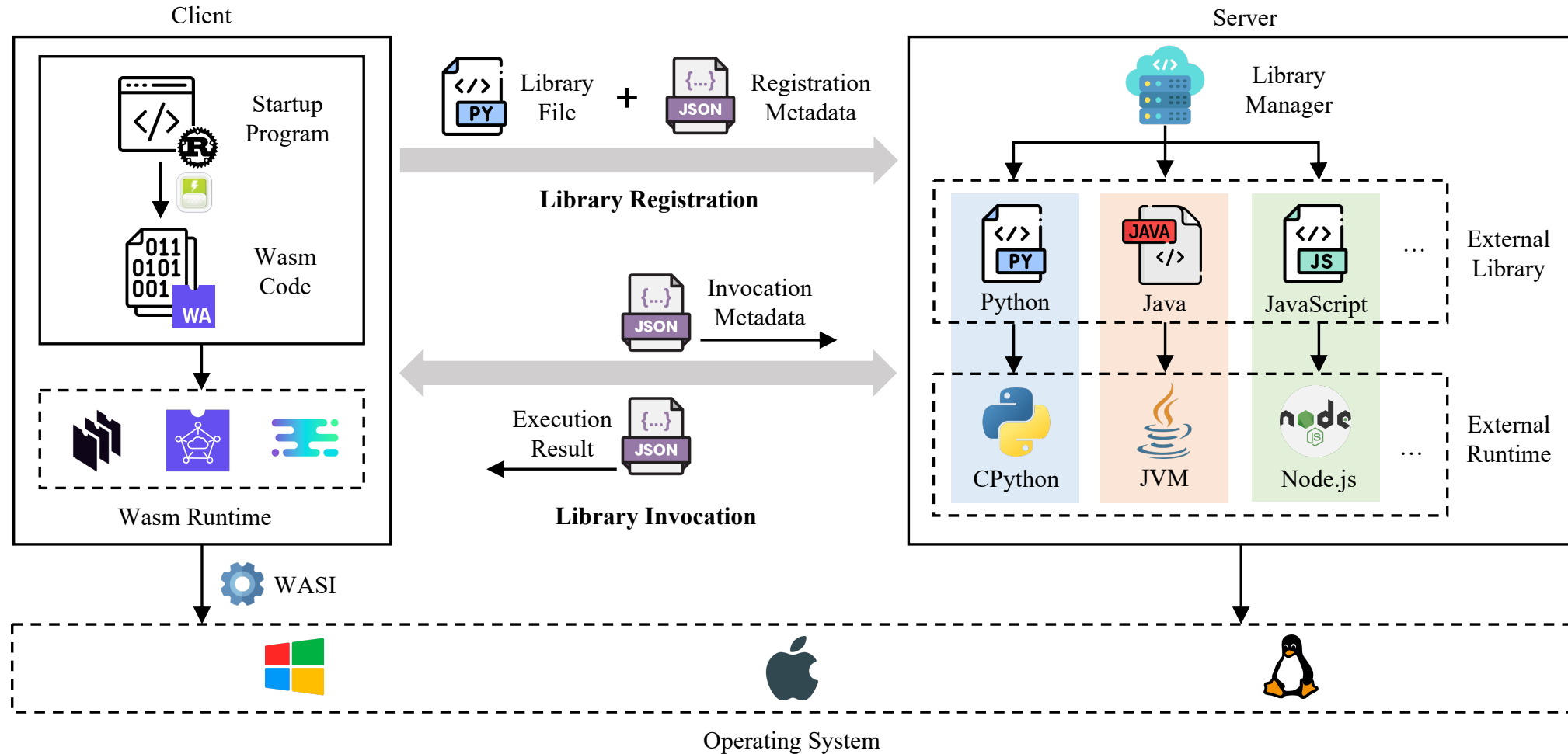
Poor Performance

Limited Support



Approach: WALL-E

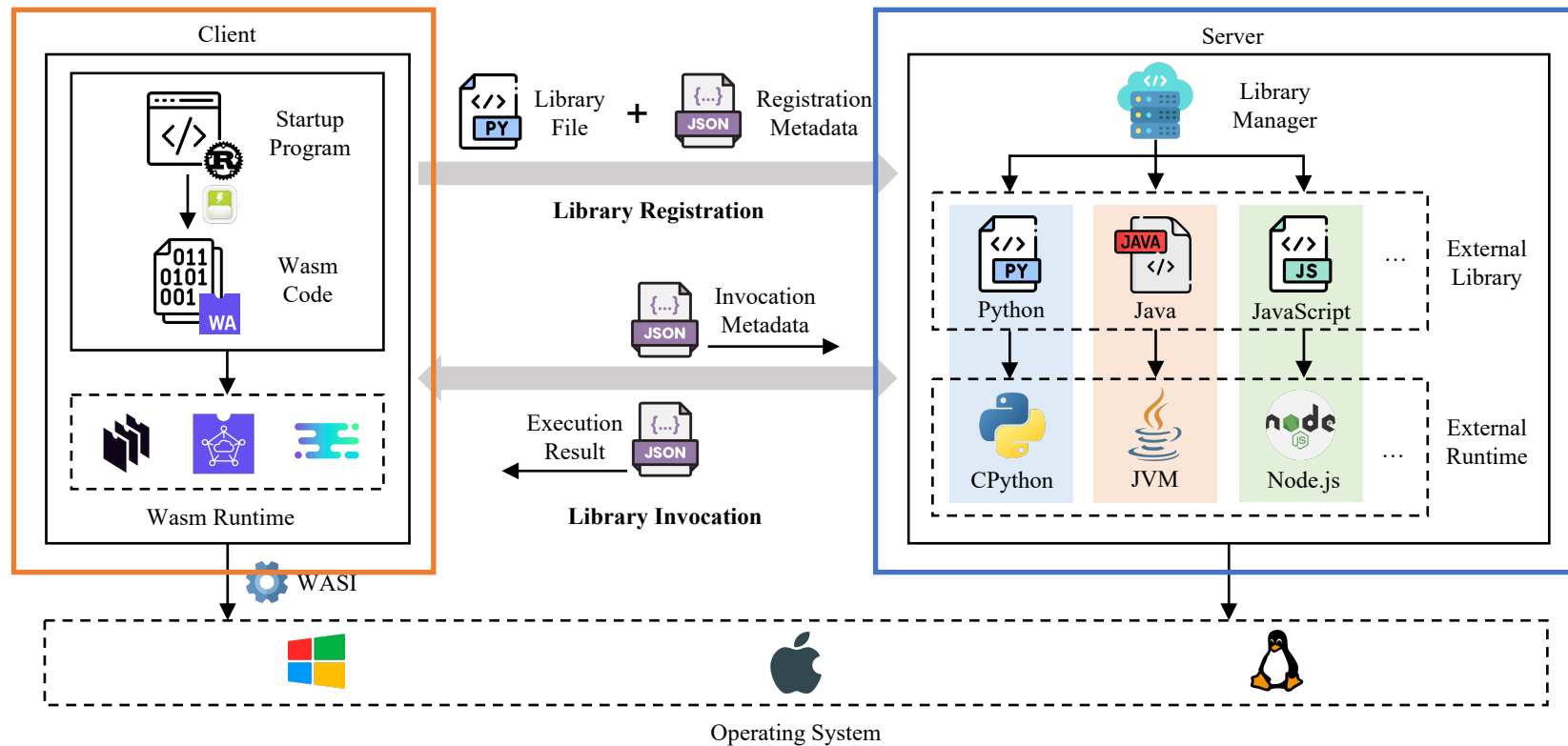
WebAssembly Language Linker via External Library



Design Principal of WALL-E

- Key idea: **External Library Linking**

- Separate the target application as an **external library**, keep it running on its original runtime
- Use a **client-server architecture** to dynamically link the external library



Extensibility

High Performance

Language Feature Compatibility

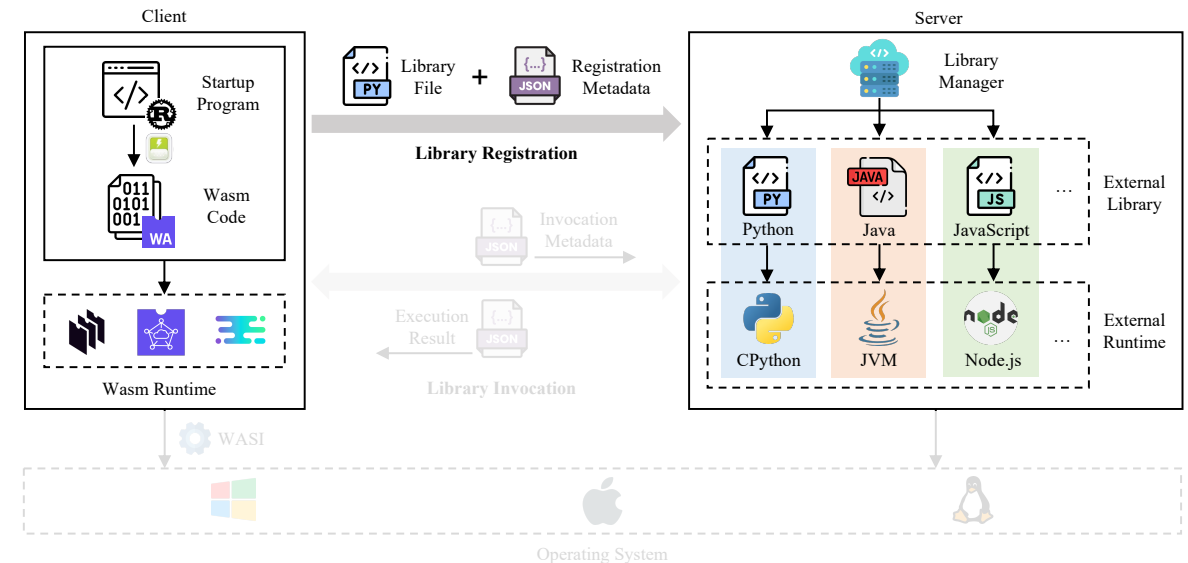


Library Registration

- A centralized mechanism for **validating** and **configuring** external libraries
- **Client-side** submission
 - Library source code
 - **Registration metadata** (JSON)

```
{
  "name": "image-processor",
  "language": "python",
  "endpoint": "auto", // Server auto-assigns or user defines
  "functions": [
    {
      "name": "enhance_contrast",
      "params": [
        {"name": "input_image", "type": "ndarray"},
        {"name": "enhancement_factor", "type": "float"}
      ],
      "return_type": "ndarray"
    }
  ],
  "dependencies": ["numpy", "opencv-python"]
}
```

- **Server-side** deployment
 - Validation
 - Resource allocation
 - Service instantiation
 - Health check



Cross-Runtime Library Invocation

- **Key challenge:** Cross-language **parameter sharing**

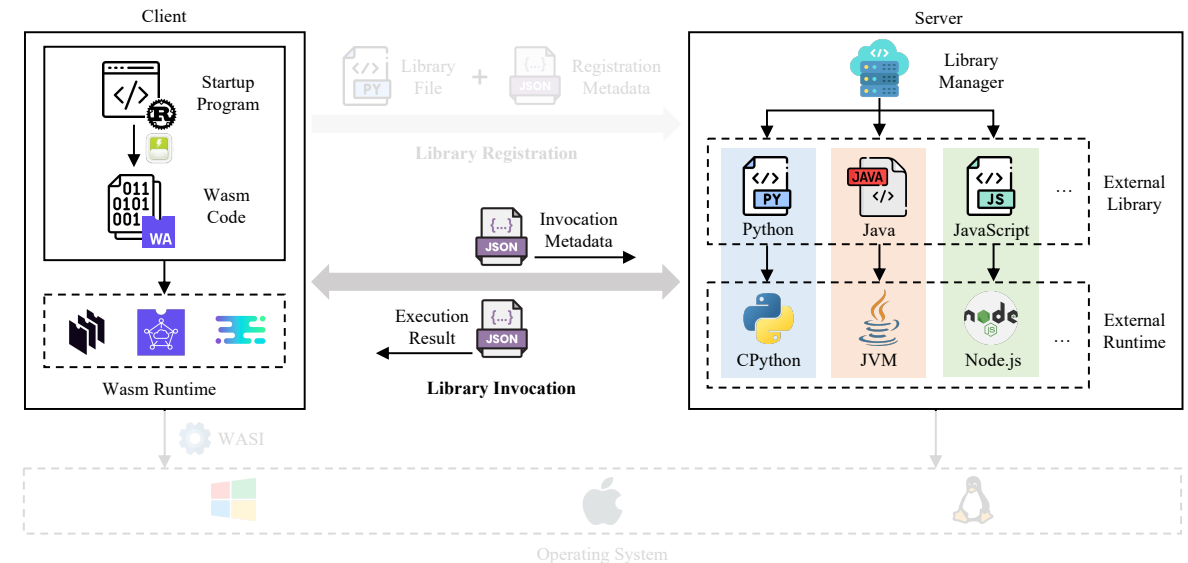
- **Client-side** Invocation

- **Invocation metadata** (JSON)
 - Target identification
 - Function specification

```
{  
  "library_id": "lib-python-a1b2c3",  
  "func": "enhance_contrast",  
  "params": [  
    {"type": "ndarray", "value": "base64_encoded_data",  
     "shape": [256, 256], "dtype": "float32"},  
    {"type": "float", "value": "1.5"}  
  ]  
}
```

- **Server-side** Processing

- Service discovery
- Request forwarding
- Distributed parameter processing
- Result response



LLM-Assisted Programming

- Provide a LLM interface for cross-language development
 - Parameter mapping
 - Dependency management
 - Error handling

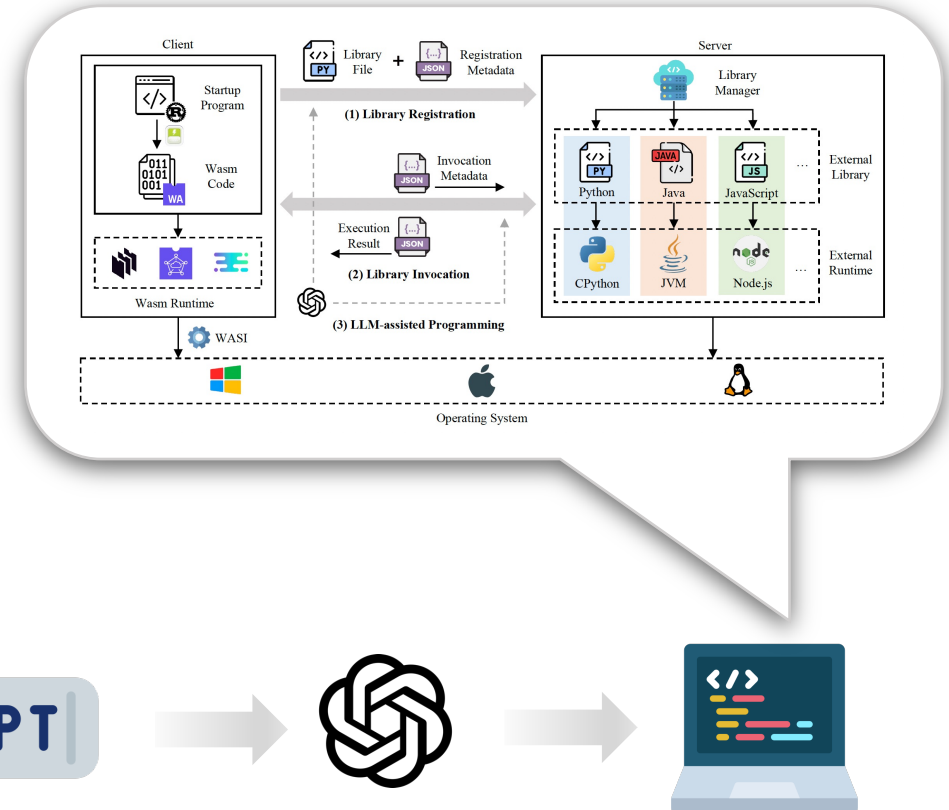
```
CROSS_LANGUAGE_PARAMETER_PROMPT = """
You are an expert in cross-language WebAssembly communication and JSON serialization.
Help me create the correct JSON parameter structure for calling external libraries.
```

```
**TARGET LIBRARY INFO:**
- Library language: {target_language}
- Library functions: {function_signatures}
```

```
**SPECIFIC PARAMETER MAPPING REQUEST:**
{parameter_mapping_request}
```

```
**ADDITIONAL CONTEXT:**
- Communication protocol: HTTP JSON API
- Parameter format: All values must be stringified
- Complex types: Use base64 encoding with shape/dtype metadata
- Use case: {use_case_description}
```

```
**RESPONSE REQUIREMENTS:**
1. Provide a complete JSON structure for parameter passing
2. Include base64 encoding instructions for binary data
3. Explain how to handle each parameter type
4. Provide validation rules for parameter formatting
5. Include error handling suggestions for malformed parameters
"""
```



PROMPT



Research Questions

RQ1 - Language Extensibility: How extensible is WALL-E in supporting various languages?

RQ2 - Execution Performance: How does the execution speed of applications using WALL-E compared to runtime nesting?

RQ3 - Communication Overhead: What is the impact of the client-server communication on the performance of WALL-E?

RQ1: Language Extensibility - Settings

- Framework: WasmEdge HTTP Service
- Target: **Top 10 managed languages** from *RedMonk Ranking*
- Language-specific benchmark: *Computer Language Benchmarks Game*

Table 1. Managed languages for the language extensibility evaluation.

Language*	Wasm Support	Runtime	Web Server	Application
JavaScript	✗	Node.js v20.16.0	Express	Tesseract.js
Python	Limited	CPython 3.12.3	Flask	Face recognition
Java	✗	OpenJDK 21.0.8	Spring Boot	Elasticsearch
PHP	Limited	PHP 8.3.10	Slim-server	Image processing
C#	Limited	.NET 8.0.117	ASP.NET Core	MsQuic
TypeScript	✗	Node.js v20.16.0	Express	Ajv validator
Ruby	Limited	CRuby 3.2.0	Sinatra	FastImage
R	✗	R 4.3.3	RestRserve	ggplot2
Scala	✗	Scala 3.5.0	Scalatra	GeoTrellis
Kotlin	✗	Kotlin 2.0.10	Javalin	Kotlin serialization

* Ranked by *The RedMonk Programming Language Rankings* [58].

Table 2. Task overview in *The Computer Language Benchmarks Game* [18].

Task	Description
binary-trees	Allocate and deallocate binary trees
fannkuch-redux	Indexed-access to tiny integer-sequence
fasta	Generate and write random DNA sequences
k-nucleotide	Hashtable update and k-nucleotide strings
mandelbrot	Generate Mandelbrot set portable bitmap file
n-body	Double-precision N-body simulation
pidigits	Streaming arbitrary-precision arithmetic
regex-redux	Match DNA 8-mers and substitute magic patterns
reverse-complement	Read DNA sequences - write their reverse-complement
spectral-norm	Eigenvalue using the power method

RQ1: Language Extensibility - Results

- Benchmark pass rate

Table 3. Benchmark pass rates on WALL-E across 10 managed languages.

Language	#Runnable	#Passed	#Failed	Pass Rate	Failure Mode
JavaScript	23	23	0	100%	-
Python	30	27	3	90%	Runtime error
Java	48	48	0	100%	-
PHP	31	22	9	70.97%	Output mismatch
C#	41	41	0	100%	-
TypeScript	10	10	0	100%	-
Ruby	31	31	0	100%	-
R	40	40	0	100%	-
Scala	14	14	0	100%	-
Kotlin	10	10	0	100%	-
Total	278	266	12	95.68%	

Answer to RQ1: WALL-E achieves an overall benchmark pass rate of 95.68% across 10 managed languages.

RQ1: Language Extensibility - Failure Analysis

- Python: Runtime error

- Benchmark: **Global** initialization for “pool”
- Web service: “pool” is **local**

```
from multiprocessing import Pool
...
def multiply_AtAv(u):
    r = range(len(u))
    tmp = pool.starmap(A_sum, zip(repeat(u), r))
    return pool.starmap(At_sum, zip(repeat(tmp), r))
...
if __name__ == "__main__":
    with Pool(processes=4) as pool:
        main(int(sys.argv[1]))
```

(a) Benchmark snippet (spectralnorm_py_4.py)

```
# Flask service routing logic
@app.route("/spectralnorm_py_4")
def call_spectralnorm_py_4():
    from python_benchmark.spectralnorm_py_4 import
        main
    from multiprocessing import Pool
    ...
    with Pool(processes=4) as pool:
        main(int(request.args.get("a")))
    return "over\n"
```

(b) Service wrapper snippet (app.py)

- PHP: Output mismatch

- Benchmark: Process management (fork/waitpid)
- Web service: fork/waitpid are **restricted**

```
function startWorker(...) {
    $pid = pcntl_fork();
    ...
}
...
function waitWorkers(&$PIDs) {
    foreach ($PIDs as $PID) {
        $pid = pcntl_waitpid($PID, $status);
    }
}
```

(a) Benchmark snippet (binarytrees_php_6.php)

```
$app->get('/binarytrees_php_6', function (...) {
    ...
    $workersPIDs = [];
    foreach ($depthIterations as $depth => $iter) {
        startWorker($depth, $iter, $workersPIDs,...);
    }
    waitWorkers($workersPIDs);
    ...
    return $response;
});
```

(b) Service wrapper snippet (server.php)

RQ2: Execution Performance - Microbenchmarks

- Baseline: python-wasmedge (runtime nesting solution)
- Benchmark: 15 python test cases
 - Compute-intensive / IO-intensive / Memory-intensive

Table 4. Execution time statistics of Python benchmarks on python-wasmedge and WALL-E (in seconds).

Compute-intensive Tasks				I/O-intensive Tasks				Memory-intensive Tasks			
BM	PY-WE*	WALL-E	↑ ×	BM	PY-WE	WALL-E	↑ ×	BM	PY-WE	WALL-E	↑ ×
pystone	96.551	0.129	748	file	0.241	0.002	120	alloc	66.194	0.184	359
nqueens	765.146	1.434	553	json	13.325	0.026	512	string	1.207	0.002	603
prime	8.332	0.010	833	text	22.534	0.050	450	large	39.322	0.080	491
matrix	70.985	0.074	959	list	84.755	0.153	553	object	118.213	0.319	370
fibonacci	1377.160	1.934	712	dict	16.701	0.043	388	recur	1.068	0.002	534
Avg.	463.635	0.716	647	Avg.	27.511	0.055	500	Avg.	45.201	0.117	386

* PY-WE refers to python-wasmedge.

RQ2: Execution Performance - Python Libraries

- jsonschema: JSON schema validation
- markdown: Markdown rendering

Table 5. Execution time statistics of real-world Python libraries (in seconds).

jsonschema ($n=1024$)				markdown ($n=256$)			
Chunk (k)	PY-WE	WALL-E	$\uparrow \times$	Chunk (k)	PY-WE	WALL-E	$\uparrow \times$
1	63.251	0.097	652	1	99.566	0.225	443
8	63.100	0.090	701	8	98.929	0.218	454
32	63.840	0.089	717	32	103.344	0.228	453
Avg.	63.397	0.092	689	Avg.	100.613	0.224	449

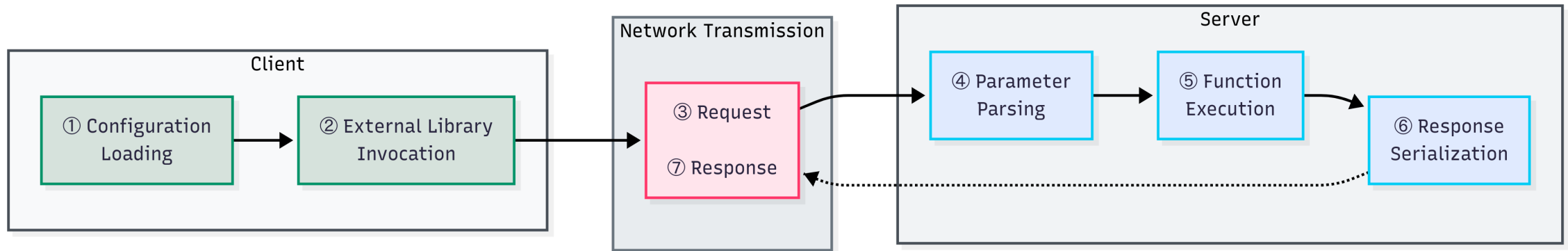
n : total inputs (docs for jsonschema, texts for markdown).

k : number of chunks/invocations (total input size n is fixed).

Answer to RQ2: WALL-E substantially outperforms python-wasmedge, achieving hundreds of speedups across both microbenchmarks and real-world Python libraries.

RQ3: Communication Overhead - Settings

- Fine-grained time analysis on the complete workflow



$\text{Overhead_client} = \text{Time}(\textcircled{1})$

$\text{Overhead_network} = \text{Time}(\textcircled{3} + \textcircled{7})$

$\text{Overhead_server} = \text{Time}(\textcircled{4} + \textcircled{6})$

$\text{Overhead_total} = \text{Overhead_client} + \text{Overhead_network} + \text{Overhead_server}$

RQ3: Communication Overhead - Results

Table 6. Communication overhead statistics across Python benchmarks (in milliseconds).

Microbenchmarks

BM	T_{total}	T_{exec}	O_{client}	O_{network}	O_{server}	O_{total}
pystone	131.725 (100%)	129.706 (98.47%)	0.134 (0.10%)	1.794 (1.36%)	0.091 (0.07%)	2.019 (1.53%)
nqueens	1442.345 (100%)	1440.165 (99.85%)	0.235 (0.02%)	1.837 (0.13%)	0.108 (0.01%)	2.180 (0.15%)
prime	13.553 (100%)	11.434 (84.37%)	0.135 (1.00%)	1.870 (13.80%)	0.114 (0.84%)	2.119 (15.63%)
matrix	78.456 (100%)	76.341 (97.30%)	0.146 (0.19%)	1.852 (2.36%)	0.117 (0.15%)	2.115 (2.70%)
fibonacci	1905.187 (100%)	1902.936 (99.88%)	0.140 (0.01%)	2.013 (0.11%)	0.108 (0.01%)	2.261 (0.12%)
Avg.	714.253 (100%)	712.116 (99.70%)	0.158 (0.02%)	1.873 (0.26%)	0.108 (0.02%)	2.139 (0.30%)

$T_{\text{total}}=T(\textcircled{1}+\textcircled{2})$, $T_{\text{exec}}=T(\textcircled{5})$, $O_{\text{client}}=T(\textcircled{1})$, $O_{\text{network}}=T(\textcircled{3}+\textcircled{7})$, $O_{\text{server}}=T(\textcircled{4}+\textcircled{6})$, $O_{\text{total}}=O_{\text{client}}+O_{\text{network}}+O_{\text{server}}$.

Table 7. Communication overhead statistics of real-world Python libraries (in milliseconds).

Python libraries

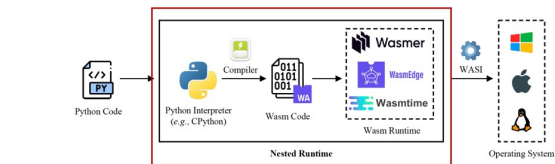
jsonschema	T_{total}	T_{exec}	O_{total}	markdown	T_{total}	T_{exec}	O_{total}
$k=1$	103.846	91.986	11.860 (11.42%)	$k=1$	226.736	221.805	4.931 (2.17%)
$k=8$	101.871	90.767	11.104 (10.90%)	$k=8$	223.549	218.172	5.377 (2.41%)
$k=32$	97.963	86.325	11.638 (11.88%)	$k=32$	222.045	217.217	4.828 (2.17%)
Avg.	101.227	89.693	11.534 (11.39%)	Avg.	224.110	219.065	5.045 (2.25%)

Answer to RQ3: WALL-E incurs low communication overhead overall. It accounts for 0.30% of total time on microbenchmarks and remains modest on real-world libraires.

Conclusion

Existing Solution: Runtime Nesting

- Compile the managed runtime into Wasm format
- Run the source program on the **Wasm-formatted runtime**

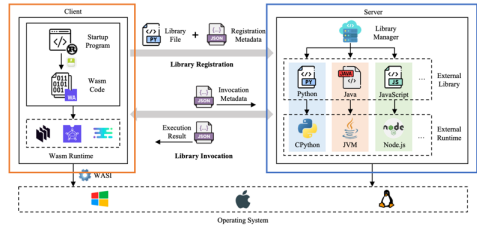


Non-extensible Poor Performance Limited Support

Background

Design Principal of WALL-E

- Key idea: **External Library Linking**
 - Separate the target application as an **external library**, keep it running on its original runtime
 - Use a **client-server architecture** to dynamically link the external library



Extensibility

High Performance

Language Feature Compatibility

Approach

RQ1: Language Extensibility - Results

- Benchmark pass rate

Table 3. Benchmark pass rates on WALL-E across 10 managed languages.

Language	#Runnable	#Passed	#Failed	Pass Rate	Failure Mode
JavaScript	23	23	0	100%	-
Python	30	27	3	90%	Runtime error
Java	48	48	0	100%	-
PHP	31	22	9	70.97%	Output mismatch
C#	41	41	0	100%	-
TypeScript	10	10	0	100%	-
Ruby	31	31	0	100%	-
R	40	40	0	100%	-
Scala	14	14	0	100%	-
Kotlin	10	10	0	100%	-
Total	278	266	12	95.68%	-

Answer to RQ1: WALL-E achieves an overall benchmark pass rate of 95.68% across 10 managed languages.

Results

Shuyao Jiang

The Chinese University of Hong Kong

syjiang21@cse.cuhk.edu.hk



Pre-print